EX: NO: 06
DATE:

HAMMING CODE

AIM:

Write a program to implement error detection and correction using hamming code concept make rest run to input data stream and verify correction.

CODE:

```
def calc_redundant_bits(m):
    for r in range(m+1):
        if (2**r)>=(m+r+1):
            return r

def main():
    data=input("Enter data:").strip()
    m=list(data)
    r=calc_redundant_bits(m)
    n=m+r
    print(f"Number of r needed: {r}")
    Print(f"Total frame length: {n}")
    Code=['0']*(n+1)
    j=0
    for i in range(1,n+1):
        if(i*(i-1))!=0:
            Code[i]=data[-(j+1)]
            j+=1
    for i in range(r):
        Parity_pos=2**i
        val=0
        for pos in range(1,n+1):
            if(pos & parity_pos)!=0 and
                pos!=parity_pos:
                if code[pos]=='1':
                    val^=1
        code[parity_pos]=str(val)
    transmitted_str="".join(code[1:][::-1])
    Printf(f"Data to be transmitted: {transmitted_str}")
    Change=input("In Transmitted bit change").strip().
                uppercase()
```

```
if change == "yes":
    received_str = input(f"Enter Corrupted data").strip()
    if len(received_str) != n:
        print(f"Error : Received data be exactly {n}
                                    bit !")
        return
    received_code = ['0'] + list(received_str[::-1])
else:
    received_code = list(code)
error position = 0
for i in range(r):
    paring_pos = 2 ** i
    val = 0
    for pos in range(1, n+1):
        if (pos & parity_pos) != 0:
            if received_code[pos] == '1':
                val ^= 1;
    if val != 0:
        error_position += parity_pos.

if error_position == 0:
    Print("\n Verification Result: No error detected")

else: print("\n Verification Result: Error detected! ")
    Print(f" Wrong bit at position : {Error_position}"),
        received_code[error_position] = '0' if received_code
        [error_position] == '1' else '1'
        corrected_str = "".join(received_code[1:][::-1])
    Print(f" Corrected data stream : {corrected_str}")

if __name__ == "__main__":
    main():
```

OUTPUT:

Enter data bits : 101100
number of redundant bits (r) needed: 4
Total frame length (m+r) : 10
Data to be transmitted : 1011100011
Transmitted bit change? (Yes/No): yes
Enter the corrupted data stream (10 bits): 1011101011
verification Result : Error detected!
Wrong bit is at position : 4
Corrected data streams: 1011100011

RESULT:

Thus the Hamming Code was implemented successfully for error detection and correction.